

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 2 – Industry Problem Solving Task

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO4 – Precision	Assessment:	Assessment Tool 2 – Industry Problem Solving Task
Weightage:	40% of CIA		
CO5 Statement:	Formulate context-free grammars, feature structures, probabilistic parsing techniques and to construct parsing frameworks. (BL-4)		
Student Name:	D.Sravanthi	Reg. No.:	192425315
Section / Batch:	2024-AI&ML	Date:	17.08.2026

Code of Conduct

I D.Sravanthi(192425315) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: D.Sravanthi

Instructions

- Read each industry scenario carefully before answering.
- Analyze the given problem from a semantic analysis perspective.
- Provide structured and logical answers with proper justification.
- Support your analysis using examples from the given scenario wherever applicable.
- Use suitable diagrams, semantic representations, logical predicates, workflows, architectures, or tables whenever necessary.
- Clearly state assumptions if additional information is required.
- Duration: 30 minutes.

Section / Topic	Focus	Question No.
Representing Meaning & Meaning Structure of Language	Analyze semantic interpretation of user queries, identify ambiguity in meaning representation, evaluate semantic processing limitations, and improve understanding in banking chatbot applications.	Q1
Syntax-Driven Semantic Analysis	Analyze multiple interpretations of spoken commands, evaluate parsing-related ambiguity, and assess semantic extraction in real-time voice assistant systems.	Q2
First-Order Predicate Calculus, Representing Linguistically Relevant Concepts, Syntax-Driven Semantic Analysis	Design a semantic processing architecture for healthcare reports, model relationships among entities and actions, represent linguistic concepts, and improve semantic extraction accuracy.	Q3

Annexure A – Industry Problem Solving Task

1. A company is developing an AI-powered customer support chatbot for banking services. The chatbot uses Context-Free Grammar (CFG) for parsing user queries. However, it faces issues such as:

Incorrect interpretation of ambiguous sentences

Failure to handle subject–verb agreement

Inefficient parsing for long queries

Example user query:

“Show me the transactions with the card from last month”

- Analyze the ambiguity present in the given sentence using CFG
- Evaluate the limitations of CFG in handling real-world conversational queries
- Evaluate and propose an improved solution using PCFG, feature structures, or Earley parsing
- Justify how your proposed method improves accuracy and efficiency in an industry setting.

Answer:

AI-Powered Customer Support Chatbot Using CFG

a) Ambiguity in the given sentence using CFG

The sentence is:

“Show me the transactions with the card from last month.”

The sentence can have more than one syntactic interpretation.

Interpretation 1 – Transactions made using the card:

The phrase **“with the card”** describes how the transactions were performed.

Show me [the transactions [with the card] [from last month]]

Meaning: Display transactions made using the card during the previous month.

Interpretation 2 – Transactions associated with a particular card:

The phrase **“with the card”** may identify which card the transactions belong to.

Show me [the transactions with the card] [from last month]

Meaning: Display transactions associated with a specific card from last month.

A basic CFG can generate both structures because CFG mainly represents grammatical relationships and does not inherently understand the user's intended meaning or banking context.

A simplified grammar could be:

$S \rightarrow VP$

$VP \rightarrow V\ NP\ PP$

$NP \rightarrow Det\ N \mid Det\ N\ PP$

$PP \rightarrow P\ NP$

$NP \rightarrow Det\ N$

$V \rightarrow Show$

Det → the

N → transactions | card | month

P → with | from

Therefore, the parser may generate multiple parse trees for the same query.

b) Limitations of CFG in real-world conversational queries

CFG is useful for representing syntactic structure, but it has several limitations in a banking chatbot:

1. **Ambiguity:** CFG can generate multiple valid parse trees without determining which meaning is intended.
2. **No probability:** All grammar rules are generally treated equally, so the most likely interpretation cannot easily be selected.
3. **Limited context awareness:** CFG does not naturally use previous conversation context.
4. **Poor handling of incomplete queries:** Users may say “last month transactions” or “show card transactions” without complete grammatical sentences.
5. **Agreement problems:** Basic CFG does not automatically enforce subject–verb or number agreement.
6. **Long-query inefficiency:** Parsing large sentences with many possible structures can become computationally expensive.
7. **Domain vocabulary:** Banking terms such as *beneficiary*, *EMI*, *overdraft*, *card limit*, and *NEFT* require additional domain-specific grammar and semantic rules.
8. **Conversational language:** Users commonly use abbreviations, spelling errors, informal expressions, and mixed-language input.

c) Improved solution using PCFG, Feature Structures, and Earley Parsing

A better solution is a **hybrid parsing architecture** combining **PCFG + Feature Structures + Earley Parsing**.

PCFG (Probabilistic CFG):

Assigns probabilities to grammar rules and ranks alternative parse trees. For example, if historical banking queries show that “with the card” usually refers to the payment instrument, that interpretation can receive a higher probability.

Feature Structures:

Grammar rules can include features such as:

Number = Singular/Plural

Person = First/Second/Third

Tense = Past/Present/Future

This helps enforce grammatical agreement.

Earley Parsing:

Earley parsing is suitable for general CFGs and can efficiently handle ambiguous and incomplete sentences. It is particularly useful for conversational systems where users may provide partial commands. The improved architecture can be:

User Query



Text/Voice Preprocessing



Tokenization + Normalization



Earley Parser



CFG Candidate Parse Trees



Feature-Structure Validation



PCFG Probability Ranking



Semantic Interpretation



Banking Entity & Intent Extraction



Banking Database/API



Response Generation

d) Justification for an industry setting

The hybrid method improves accuracy because **PCFG selects the most probable interpretation**, while **feature structures eliminate grammatically inconsistent parses**. Earley parsing allows the system to handle ambiguity and incomplete queries more effectively.

For example, the chatbot can identify:

Intent = SHOW_TRANSACTIONS

Instrument = CARD

Time_Range = LAST_MONTH

This structured representation can then be sent to the banking transaction service.

The approach also improves efficiency because the parser can prioritize likely interpretations instead of treating every possible parse equally. In a banking environment, this reduces incorrect transaction

searches, improves response accuracy, supports natural conversational queries, and provides a more reliable customer experience.

Conclusion: A hybrid **Earley + PCFG + Feature Structure** approach is more suitable than a basic CFG for an industrial banking chatbot because it combines syntactic flexibility, probabilistic ambiguity resolution, grammatical validation, and efficient parsing.

2. A voice assistant system (like Alexa/Siri-type applications) processes spoken commands using parsing techniques. The system currently uses top-down parsing, but faces:

Backtracking issues

Poor handling of incomplete or ambiguous input

Difficulty in real-time response

Example command:

“Book a flight to Delhi with a window seat”

- a) Analyze how the given sentence can lead to multiple parse structures
- b) Analyze the limitations of top-down parsing in this real-time application
- c) Analyze how Earley parsing can improve handling of ambiguity and partial inputs
- d) Compare and analyze the performance of top-down vs Earley parsing for such industry use cases.

Answer:

Voice Assistant Using Top-Down and Earley Parsing

a) Multiple parse structures for the given command

The command is:

“Book a flight to Delhi with a window seat.”

The phrase **“with a window seat”** can be interpreted in different ways.

Interpretation 1 – Seat preference:

Book

└─ a flight
 └─ to Delhi
 └─ with a window seat

Meaning: Book a flight to Delhi and request a window seat.

Interpretation 2 – Flight description:

The phrase may initially be attached to the flight itself:

Book

└─ a flight
 └─ to Delhi
 └─ with a window seat

The second interpretation is less natural in the real-world domain, but syntactically a parser may consider it.

There may also be ambiguity about whether “**Delhi**” refers to the destination airport/city and whether “**window seat**” is a booking constraint.

Therefore, syntactic parsing alone is insufficient. The voice assistant must combine syntax with **semantic and domain information**.

b) Limitations of top-down parsing in a real-time application

Top-down parsing starts from the start symbol and attempts to generate a structure that matches the input.

Its major limitations include:

1. **Backtracking:** When a selected grammar rule fails, the parser may return to an earlier decision and try another rule.
2. **Repeated work:** Similar partial structures may be examined multiple times.
3. **Ambiguity handling:** Multiple possible interpretations can result in extensive search.
4. **Left-recursion problems:** Some top-down parsers can enter infinite recursion when grammar rules contain left recursion.
5. **Partial input difficulty:** Spoken commands may arrive incrementally, making incomplete sentences difficult to process.
6. **Real-time constraints:** Excessive backtracking can increase response latency.
7. **Speech recognition errors:** Voice input may contain missing or incorrectly recognized words, increasing parsing failures.

For a voice assistant, even a small delay can negatively affect the user experience.

c) How Earley parsing improves ambiguity and partial-input handling

Earley parsing is a dynamic-programming parsing technique capable of handling **general CFGs**, including ambiguous grammars.

It uses three major operations:

- **Prediction:** Predicts possible grammar structures.
- **Scanning:** Matches grammar symbols with input words.
- **Completion:** Completes partially recognized structures.

For example, when the system receives:

“Book a flight to Delhi...”

Earley parsing can maintain a partial parse while waiting for additional input.

When the next words arrive:

“...with a window seat”

the parser can extend the existing structure rather than restarting the complete parsing process.

This is valuable for voice assistants because spoken commands may be processed incrementally.

Earley parsing can also retain multiple possible interpretations until enough information is available to select the appropriate one.

d) Top-down vs Earley parsing

Factor	Top-Down Parsing	Earley Parsing
Parsing strategy	Starts from grammar start symbol	Uses prediction, scanning and completion
Ambiguity	Difficult and may require backtracking	Handles ambiguity naturally
Backtracking	Often required	Reduced through chart/dynamic programming
Partial input	Relatively difficult	Better suited
General CFG support	Depends on implementation	Yes
Left recursion	Can be problematic	Can handle it
Real-time response	Can suffer from delays	Better for complex/ambiguous inputs
Memory usage	Generally lower	Higher because chart states are stored
Long/complex sentences	Can become inefficient	More systematic
Voice-assistant suitability	Moderate	High

For a simple command, top-down parsing may be fast and easy to implement. However, for an industrial voice assistant handling **ambiguous, incomplete, noisy, and continuously arriving speech**, Earley parsing provides greater robustness.

Conclusion: Earley parsing is a better choice for complex voice-assistant applications because it can maintain partial parses, manage ambiguity, reduce unnecessary backtracking, and work with general CFGs. In practice, it should be combined with speech recognition, semantic analysis, and domain-specific intent classification.

3. A healthcare company is developing an AI system to analyze patient medical reports and automatically extract critical information such as diagnosis, prescribed treatments, and follow-up actions.

The system must handle:

Complex and lengthy medical sentences

Ambiguity in sentence structure

Accurate subject-verb agreement

Relationships between medical actions and entities (sub-categorization)

Real-time processing for hospital use

Example input:

"The doctor who reviewed the patient last week recommends starting medication and scheduling a follow-up visit in Chennai."

The current system, based only on basic Context-Free Grammar (CFG), fails to correctly interpret relationships and produces inconsistent outputs.

a) Design a complete NLP architecture integrating:

CFG for syntactic structure

Probabilistic CFG (PCFG) for ambiguity resolution

Feature Structures for agreement handling

Efficient parsing techniques

b) Develop a step-by-step workflow showing how the system processes the given medical sentence from input to structured output (diagnosis, action, location).

c) Create a method to:

Incorporate sub-categorization frames for medical verbs

Handle real-time data processing

Improve accuracy and scalability in a hospital environment .

Answer:

NLP Architecture for AI-Based Healthcare Report Analysis

The example sentence is:

"The doctor who reviewed the patient last week recommends starting medication and scheduling a follow-up visit in Chennai."

The system must identify the doctor as the subject, understand the relative clause, identify the recommended medical actions, and associate Chennai with the follow-up visit.

a) Complete NLP Architecture

A suitable architecture is:

Medical Report / Clinical Text



Text Preprocessing



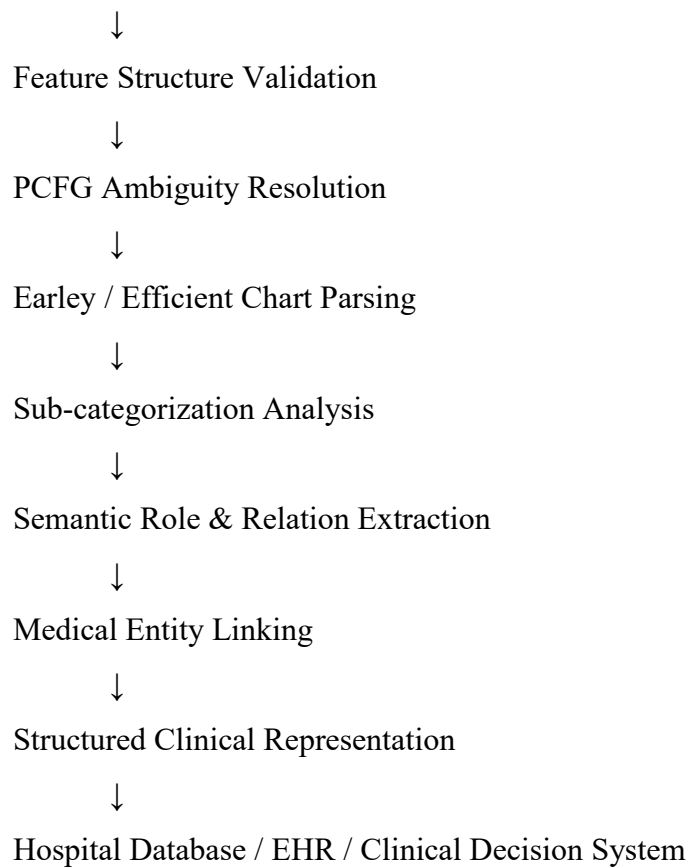
Sentence Segmentation & Tokenization



Medical NER



CFG Syntactic Parsing



1. Preprocessing

The system first performs:

- Sentence segmentation
- Tokenization
- Spelling and abbreviation normalization
- Medical terminology normalization

For example:

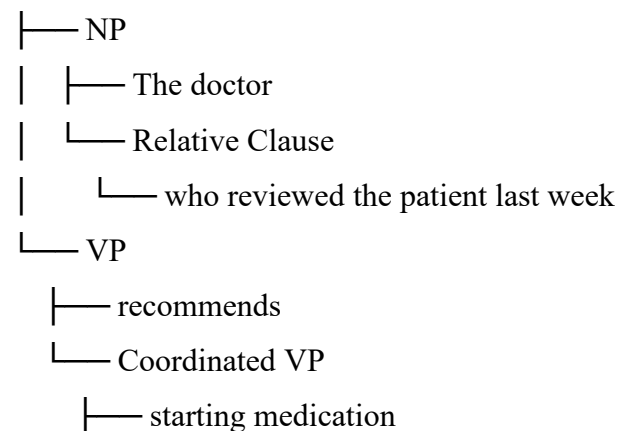
“follow-up visit” → medical follow-up action

2. CFG for syntactic structure

CFG identifies the basic grammatical structure.

A simplified structure is:

S



└─ scheduling a follow-up visit in Chennai

This establishes the relationships between subjects, verbs, objects, and modifiers.

3. PCFG for ambiguity resolution

Medical sentences can have several possible parse structures. PCFG assigns probabilities to alternative parses based on training data.

For example, the system can determine that:

“in Chennai”

is more likely to modify “**follow-up visit**” rather than “**scheduling**” or “**medication.**”

Thus, the most probable medically meaningful interpretation is selected.

4. Feature Structures for agreement

Feature structures represent grammatical properties such as:

Person = Third

Number = Singular

Tense = Present

For example:

Doctor:

Number = Singular

Person = Third

Recommends:

Number = Singular

Person = Third

The system can therefore verify that “**doctor recommends**” is grammatically consistent.

5. Efficient parsing

Earley parsing or another chart-based parser can be used to efficiently handle long and complex sentences. Previously computed partial structures can be reused instead of repeatedly parsing the same parts.

b) Step-by-step workflow for the given sentence

Step 1 – Input

The doctor who reviewed the patient last week
recommends starting medication and scheduling a
follow-up visit in Chennai.

Step 2 – Tokenization

The sentence is divided into tokens:

The | doctor | who | reviewed | the | patient |
last | week | recommends | starting | medication |

and | scheduling | a | follow-up | visit | in | Chennai

Step 3 – Medical Named Entity Recognition

The system identifies important entities:

DOCTOR → doctor

PATIENT → patient

MEDICATION → medication

LOCATION → Chennai

FOLLOW-UP → follow-up visit

Step 4 – CFG parsing

The syntactic parser identifies:

Main Subject = The doctor

Relative Clause = who reviewed the patient last week

Main Verb = recommends

Action 1 = starting medication

Action 2 = scheduling a follow-up visit

Location = Chennai

Step 5 – Feature checking

The system checks agreement:

Subject = doctor

Number = Singular

Verb = recommends

Number = Singular

Therefore:

doctor + recommends ✓

Step 6 – PCFG ambiguity resolution

The parser generates possible interpretations and assigns probabilities.

The preferred interpretation is:

The doctor reviewed the patient last week and recommends two actions: starting medication and scheduling a follow-up visit in Chennai.

Step 7 – Sub-categorization analysis

The verb “**recommends**” expects an action or proposition as its complement.

A suitable frame is:

recommend(Agent, Action)

Therefore:

recommend(

Agent = doctor,

Action = starting medication

)

recommend(

Agent = doctor,

Action = scheduling follow-up visit

)

Step 8 – Semantic relation extraction

The system extracts:

Doctor

↓ recommends

Starting Medication

Doctor

↓ recommends

Scheduling Follow-up Visit

↓

Location

↓

Chennai

Step 9 – Structured output

A possible structured representation is:

```
{
  "subject": "doctor",
  "previous_action": "reviewed patient",
  "previous_time": "last week",
  "recommendations": [
    {
      "action": "start medication"
    },
    {
      "action": "schedule follow-up visit",
      "location": "Chennai"
    }
  ]
}
```

This structured information can then be stored in or sent to the hospital's Electronic Health Record system.

c) Method for Sub-Categorization, Real-Time Processing, Accuracy and Scalability

1. Incorporating sub-categorization frames

Medical verbs should be associated with predefined semantic frames.

Examples:

recommend(Doctor, Treatment/Action)

prescribe(Doctor, Medication, Patient)

diagnose(Doctor, Disease, Patient)

schedule(Staff/Doctor, Appointment, Patient, Location)

administer(HealthcareWorker, Medication, Patient)

For example:

“The doctor recommends starting medication.”

```
recommend(  
    Agent = Doctor,  
    Complement = Treatment_Action  
)
```

This prevents the system from incorrectly treating “**starting medication**” as an ordinary noun phrase.

A medical lexicon can store these frames together with synonyms and domain-specific terminology.

2. Real-time processing

For hospital environments, the system should use:

- Incremental parsing
- Earley/chart parsing
- Streaming NLP pipelines
- Cached grammar rules
- Parallel processing
- Lightweight medical NER
- Asynchronous database communication
- Priority queues for urgent clinical reports

Instead of waiting for an entire large report, the system can process sentences incrementally as they arrive.

3. Improving accuracy

Accuracy can be improved by combining several techniques:

CFG: Provides syntactic structure.

PCFG: Selects the most probable interpretation.

Feature Structures: Enforces grammatical agreement.

Medical NER: Identifies diseases, medicines, doctors, patients, dates, and locations.

Sub-categorization: Determines how medical verbs relate to their arguments.

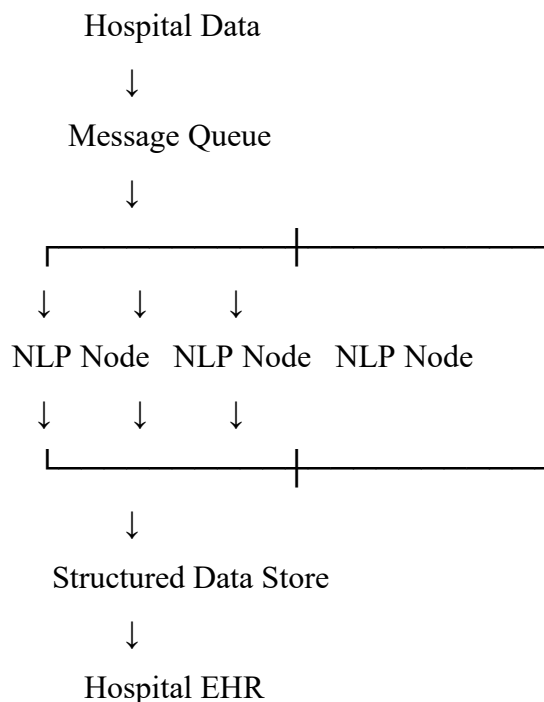
Medical ontology: Connects terms to standardized medical concepts.

Context analysis: Uses surrounding sentences and previous clinical information.

Human verification: Flags uncertain or high-risk interpretations for review by healthcare professionals.

4. Improving scalability

A hospital may process thousands of medical reports daily. Therefore, the architecture should support scalability through:



Multiple NLP processing nodes can work simultaneously. A message queue distributes incoming reports, while cached grammar models and medical dictionaries reduce repeated computation.

Overall Industry Recommendation

A basic CFG alone is not sufficient for a healthcare NLP system because medical language is complex, ambiguous, domain-specific, and time-sensitive.

A **hybrid architecture** is therefore recommended:

CFG

+

PCFG

+

Feature Structures

+

Earley/Chart Parsing

+

Medical NER

+

Sub-categorization Frames

+

Medical Ontology

+

Semantic Role/Relation Extraction

This combination provides **syntactic correctness, probabilistic ambiguity resolution, agreement checking, semantic understanding, and efficient processing**. It is therefore better suited for real-time hospital applications where accurate extraction of diagnoses, treatments, follow-up actions, and locations is critical.

Rubrics for Calculation

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding ; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

Q. No. / Criteria Level	Understanding of Industry Problem	Application of Engineering Concepts	Solution Design	Use of Tools	Analysis	Professionalism	Sub. Total	Total %
1								
2								
3								

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

